INFLUENCER ANALYTICS DASHBOARD

TECHNICAL DOCUMENTATION

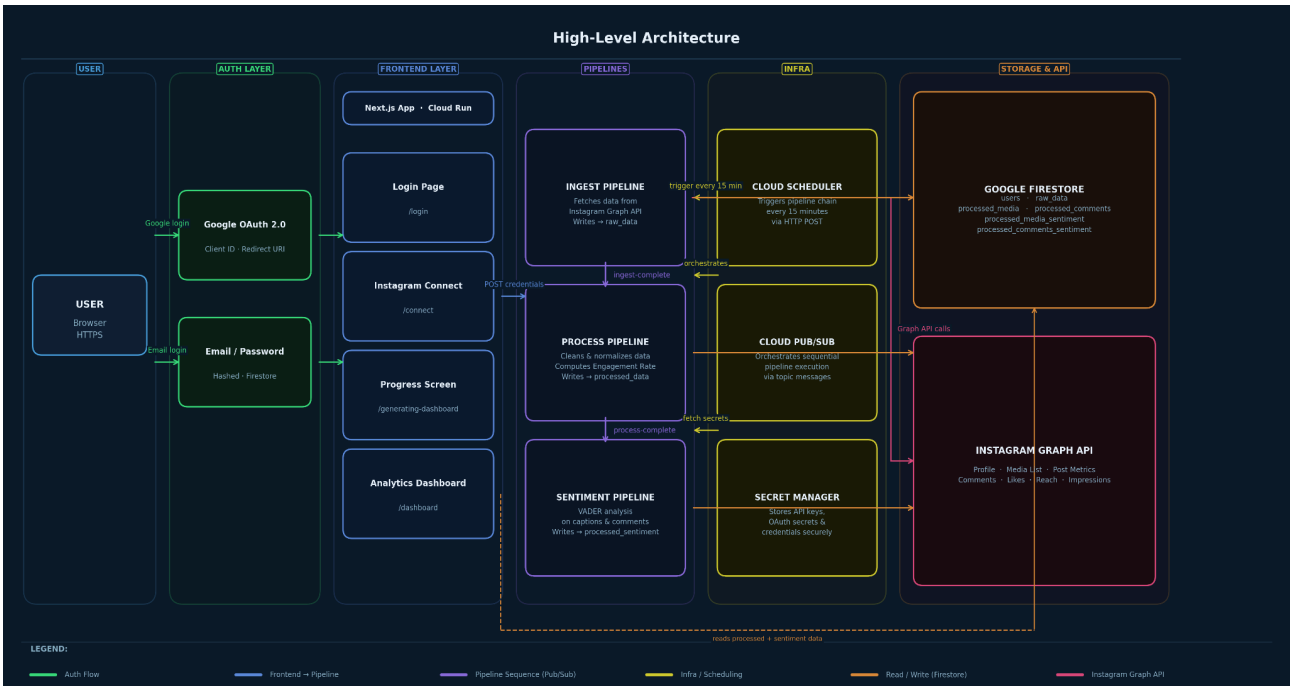
TABLE OF CONTENTS

1. System Overview
2. High-Level Architecture
3. Technology Stack
4. Authentication System
    - 4.1 Google OAuth 2.0
    - 4.2 Email & Password Authentication
5. Backend Pipeline Architecture
    - 5.1 Ingest Pipeline
    - 5.2 Process Pipeline
    - 5.3 Sentiment Analysis Pipeline
    - 5.4 Pipeline Orchestration & Scheduling (Pub/Sub + Cloud Scheduler)
    - 5.5 Secret Manager Integration
6. Firestore Data Model
7. Frontend Architecture: Next.js
8. Cloud Run Deployment
9. API Credentials & Environment Variables
10. Security Considerations
11. Technical System Flow Summary

1. System Overview

The Influencer Analytics Dashboard is a cloud-native web application built on Google Cloud Platform. It consists of a Next.js frontend application and three independent Python-based backend pipeline services, all containerized and deployed on Google Cloud Run. Data is stored in Google Firestore and fetched from the Instagram Graph API. Authentication is managed via Google OAuth 2.0 and a custom email/password flow.

2. High-Level Architecture



3. Technology Stack

Layer	Technology
Frontend	Next.js (React)
Backend Pipelines	Python
Sentiment Analysis	VaderSentiment
Database	Google Firestore
Authentication	Google OAuth 2.0 + Custom Email/Password
Deployment	Google Cloud Run
External API	Instagram Graph API
Containerization	Docker

4. Authentication System

4.1 Google OAuth 2.0

The platform uses a Google OAuth Client ID configured in the Google Cloud Console. The following are configured:

- Authorized JavaScript Origins, the deployed Cloud Run frontend URL and localhost for development
- Authorized Redirect URIs, the callback endpoints that Google redirects to after successful authentication

Flow:

1. User clicks "Continue with Google" on the Login page
2. Google OAuth consent screen appears
3. On approval, Google returns an authorization code to the redirect URI
4. The platform exchanges this for an access token and retrieves the user's name and email from Google
5. A session is created and the user is redirected to the Instagram Connect page

4.2 Email & Password Authentication

Users can register with a name, email, and password. Passwords are stored securely (hashed) in Firestore. On login, the submitted password is verified against the stored hash before a session is created.

5. Backend Pipeline Architecture

Three independent services run sequentially once Instagram credentials are submitted. Each is a containerized Python service deployed on Cloud Run.

5.1 Ingest Pipeline

This pipeline connects to the Instagram Graph API using the user-provided access_token, username, and user_id. It fetches raw data including profile info, media list, post metrics (likes, comments, reach, impressions), and individual comment text, etc.

Output: Raw JSON documents written to the raw_data collection in Firestore.

Key API Endpoints Used from Instagram Graph API:

- GET /{user-id} — profile fields
- GET /{user-id}/media — list of posts
- GET /{media-id} — individual post metrics
- GET /{media-id}/comments — comments on a post

5.2 Process Pipeline

This pipeline reads raw data from the raw_data Firestore collection, cleans and structures it like handles null values, normalizes timestamps, calculates derived metrics like engagement rate, and prepares a structured schema. Some formulas are also used to calculate different parameters with raw data like below given:

Metric	Formula	Applied On
Engagement Rate	((Likes + Comments) / Followers) × 100	Per Post
Viral Potential Score	((Saves × 3) + (Comments × 2) + Likes) / Reach	Per Post
Hook Efficiency	1 − Reels Skip Rate	Reels Only
Rewatch Ratio	Replays / Total Plays	Reels Only
Ghost Score	(Reach / Followers) × 100	Profile Level
Traffic Velocity	(Website Clicks / Profile Views) × 100	Profile Level
Reach Click %	(Website Clicks / Reach) × 100	Profile Level
Average Sentiment Score	(Sum of VADER Compound Scores / Total Texts) × 100	All Comments & Captions

Output: Structured or processed documents are saved into different collections/docs in Firestore and are named accordingly. For example, *processed_media* stores processed media data. The collections include: processed_comments, processed_comments_sentiment, processed_comments_sentiment_summary, processed_media, processed_media_sentiment, processed_profile, metadata and raw_data.

5.3 Sentiment Analysis Pipeline

This pipeline reads processed data, extracts post captions and comment text, and runs VADER sentiment analysis on each. It returns a compound score for each text unit.The Score Interpretation are as follows:

- compound >= 0.05 → Positive
- compound <= -0.05 → Negative
- -0.05 < compound < 0.05 → Neutral

Output: Sentiment scores and labels appended to documents in the firestore collection as *processed_sentiment* documents.

5.4 Pipeline Orchestration and Scheduling (Pub/Sub + Cloud Scheduler)

Cloud Scheduler is set to run the pipeline chain after every 15 minutes through an HTTP POST request to the Cloud Run endpoint of the Ingest Pipeline. This makes sure that data is automatically updated at all times.

Cloud Pub/Sub coordinates the execution of the pipelines in a sequence. The completion message is sent by each pipeline to its corresponding Pub/Sub topic when the pipeline is finished and this message causes the next pipeline in the chain to become subscribers.

Topics & Subscriptions:

Pub/Sub Topic	Publisher	Subscriber Triggered
ingest-complete	Ingest Pipeline	Process Pipeline
process-complete	Process Pipeline	Sentiment Pipeline

5.5 Secret Manager Integration

All credentials that are sensitive are stored in Google Secret Manager and accessed by each service on-demand. None of the secrets are programmed in the source code or Docker images.

Secrets stored:

Secret Name	Used By
instagram-access-token	Ingest Pipeline
google-oauth-client-secret	Next.js App
firestore-service-account	All Pipeline Services

Access Pattern: Every Cloud Run service is configured with a GCP Service Account with IAM role roles/secretmanager.secretAccessor which only allows it to request only the secrets required at startup or request time.

6. Firestore Data Model

a. Collection: users

This collection stores authentication and profile-related information for each user.

```
users/{user_email}
  ├── created_at: timestamp
  ├── dashboard_access: boolean
  ├── user_email: string
  ├── image: string (profile image URL)
  ├── instagram_credentials: map
  ├── last_login: timestamp
  ├── user_name: string
  ├── provider: "google" | "email"
  └── updated_at: timestamp
```

b. Collection: users_data

Each document is named using the user's email. This collection stores all structured and processed data related to that specific user.

```
users_data/{user_email}
  ├── metadata: map
  ├── processed_comments: array/map
  ├── processed_comments_sentiment: map
  ├── processed_comments_sentiment_summary: map
  ├── processed_media: array/map
  ├── processed_media_sentiment: map
  ├── processed_profile: map
  └── raw_data: map
```

All processed and structured documents for a specific user are stored inside their respective {user_email} document in this collection.

c. Collection: user_pipeline_status

Each document is named using the user's email and stores the pipeline processing status.

```
user_pipeline_status/{user_email}
  └── pipeline_status: map
      ├── completed_at: timestamp
      ├── error_message: string | null
      ├── failed_at: timestamp | null
      ├── ingest_started_at: timestamp
      ├── ingest_completed_at: timestamp
      ├── ingest_status: "pending" | "processing" | "completed" | "failed"
      ├── message: string
      ├── progress: number
      ├── progress_percentage: number
      ├── records_ingested: number
      ├── stage: string
      └── updated_at: timestamp
```

The Firestore database contains three main collections:

1. users → Login & profile details
2. users_data → Structured + processed + raw Instagram data
3. user_pipeline_status → Data processing pipeline tracking

7. Frontend Architecture (Next.js)

The frontend is a Next.js application with four primary pages/routes:

a) **> / or /login — Login Page:** It handles both Google OAuth initiation and email/password form submission. It manages auth state and redirects authenticated users directly to /connect.

b) **/connect — Instagram Connect Page:** It is a protected/private route. It presents a form for access_token, username, and user_id. On submission, it triggers the backend pipeline chain and redirects to /generating-dashboard.

c) **/generating-dashboard — Pipeline Progress Page:** It polls the backend or Firestore to check pipeline completion status. It displays a step-by-step progress UI. Once all pipelines are complete, it auto-redirects to /dashboard.

d) **/dashboard — Analytics Dashboard Page:** It is also a protected route. It fetches processed and sentiment data from Firestore and renders charts, KPI cards, and sentiment breakdowns. It displays three different tabs: Home, Dashboard and Insights, as each provides a level of analytics to a different degree.

i) The Home tab displays top-level KPI cards and summary charts which are fetched using the processed and sentiment Firestore collections.

ii) The Dashboard tab makes detailed post-level analytics such as post-engagement, reach, impressions, and sentiment scores.

iii) The Insights tab shows enhanced audience persona and demographics information. The Audience Persona component is conditional because it is rendered as a result of eligibility validation carried out within *InsightsPage.jsx*. The component first reads profile data and verifies two conditions and after that any call to the API is made:

- *account_type* should be Business or Creator.
- *followers_count* should be 100 and more.

In case of failure of either of the above conditions in the linked account, the tab will show a locked mode with the actual account type and number of followers of the user alongside the eligibility criteria. As the two requirements are satisfied, complete audience demographics are automatically fetched and shown using */api/audience-insights* endpoint.

8. Cloud Run Deployment

All services are containerized using Docker and deployed on Google Cloud Run.

Service	Type	Trigger
ingest-pipeline	Cloud Run Service	HTTP POST from frontend
process-pipeline	Cloud Run Service	HTTP POST (after ingest completes)
sentiment-pipeline	Cloud Run Service	HTTP POST (after process completes)
nextjs-app	Cloud Run Service	Public HTTPS — serves the frontend

General Cloud Run Configuration:

- Region: (your GCP region, e.g., us-central1 or asia-south1)
- Authentication: Allow unauthenticated requests for the frontend; pipeline services can be internal-only
- Auto-scaling: Minimum 0 instances (scale to zero when idle), Maximum as per load requirements

9. API Credentials & Environment Variables

The following environment variables are required across services:

For Next.js App:

GCP_PROJECT_ID=<your-gcp-project-id>

NEXT_PUBLIC_GOOGLE_CLIENT_ID = <your-google-oauth-client-id>

NEXT_PUBLIC_GOOGLE_CLIENT_SECRET = <your-google-oauth-client-secret>

NEXT_PUBLIC_INGEST_CLOUD_URL = <your-cloud-run-ingest-service-url>

NEXTAUTH_URL = <your-app-base-url>

NEXTAUTH_SECRET = <your-nextauth-secret>

GOOGLE_REDIRECT_URI = <your-google-redirect-uri>

TOKEN_ENCRYPTION_KEY = <your-token-encryption-key> OR

ENCRYPTION_KEY=<your-encryption-key>

FIREBASE_PROJECT_ID = <your-firestore-project-id>

For Pipeline Services:

GCP_PROJECT_ID = <your-gcp-project-id>

GOOGLE_APPLICATION_CREDENTIALS = <path-to-service-account-json>

ENCRYPTION_KEY = <your-encryption-key>

INSTAGRAM_GRAPH_API_BASE = https://graph.instagram.com

10. Security Considerations

a. **Authentication:** Google OAuth tokens are short-lived and handled by Google's secure infrastructure. Email/password credentials are hashed before storage like plain-text passwords are never stored.

b. **Instagram Access Token:** The access token provided by the user is used only during the ingest pipeline execution and stored in Firestore. It should ideally be encrypted at rest.

c. **Cloud Run Services:** Backend pipeline services should be configured to accept requests only from the frontend service (using Cloud Run service-to-service authentication with IAM) rather than being publicly exposed.

d. **Firestore Rules:** Firestore security rules should be configured so that each user can only read/write their own documents which is being enforced via the userId document path pattern.

e. **Insights tab (Audience Persona Validation):** It uses Client-Side Eligibility on the page. This ensures that unnecessary API access is not used and to guard against unauthorized account_type and followers_count data retrieval requests, both metrics should be validated against the profile data before a network request is sent.

11. System Flow Summary